COMP90038 Algorithms and Complexity

Decrease-and-Conquer-by-a-Factor

Junhao Gan

Week 5 Lecture 10

Semester 1, 2026

Decrease-and-Conquer Again

The previous lecture looked at a problem solving approach that went like this:

To solve a problem of size n, try to express the solution in terms of a solution to the same problem of size $n - 1$.

A simple example was topological sorting on a DAG $G = \langle V, E \rangle$:

1. find a node u with no incoming edge;
2. append u to the current topological order;
3. remove u and all its incident edges from the graph;
4. compute and append the topological order of the resulted graph (with $|V| - 1$ nodes) after u;

Decrease-and-Conquer by a Factor

We now look at better utilization of the approach, often leading to methods with logarithmic time behaviour or better!

Decrease-by-a-constant-factor is exemplified by *binary search*.

Let us look at these and other instances.

Binary Search

Consider an array A which is sorted in non-decreasing order.

Start by comparing the target key k against the middle element $A[m]$ in the given sorted array.

If $A[m] = k$, we are done.

If $A[m] < k$, search the sub-array up to $A[m-1]$ recursively.

If $A[m] > k$, search the sub-array from $A[m+1]$ recursively.

$$\underbrace{A[0] \ \cdots \ A[m-1]}_{\text{search here if } A[m] > k} \quad \overset{= k?}{\underset{\downarrow}{A[m]}} \quad \underbrace{A[m+1] \ \cdots \ A[n-1]}_{\text{search here if } A[m] < k}$$

Complexity of Binary Search

The worst-case situation for binary search is when the target k is not in the array. We have the following recursive equation for the cost, in terms of the size n of the array:

$$C(n) = \begin{cases} 1 & \text{if } n = 1 \\ C(\lfloor n/2 \rfloor) + 1 & \text{if } n > 1 \end{cases}$$

A closed form for this is

$$C(n) = \lfloor \log_2 n \rfloor + 1$$

The worst-case time complexity is $O(\log n)$.

In the worst case, searching for k in an array of size 1,000,000 requires 20 comparisons.

Large Power Computation

Recall the problem that computing a^n for a positive integer $a > 0$ and an integer $n \geq 0$.

We can solve the problem by the following algorithm:

- if $n = 0$, return 1;
- if n is *even*, recursively compute $a^{n/2}$, and then return $a^{n/2} \cdot a^{n/2}$;
- if n is *odd*, recursively compute $a^{\frac{n-1}{2}}$, and then return $a^{\frac{n-1}{2}} \cdot a^{\frac{n-1}{2}} \cdot a$;

Since in each iteration, the exponent n is halved, and there are at most two multiplications, the total running time is thus bounded by $O(\log n)$.

The k-Selection Problem

Given an array A of n integers and a specified integer $1 \leq k \leq n$, the goal of the *k-Selection* problem is to **return** the *k^{th} smallest* integer in A.

Some simple cases:

- if $k = 1$ or $k = n$, the problem can be solved by scanning A once;
- if A is sorted, the k^{th} smallest integer can be returned in $O(1)$ time;

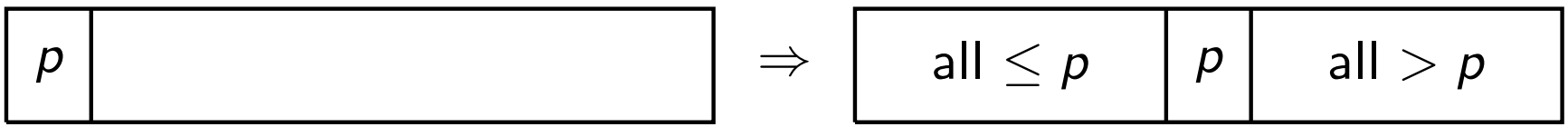
While we have already known that sorting A can be done in $O(n^2)$ time by Selection Sort, as we will see later this semester, there exist algorithms for sorting A in $O(n \log n)$ time.

Nonetheless, sorting the array seems like an *overkill*.

A Detour via Partitioning

Partitioning an array A with some *pivot* element p means reorganizing the array so that:

all elements to the left of p are no greater than p, while those to the right are strictly greater.



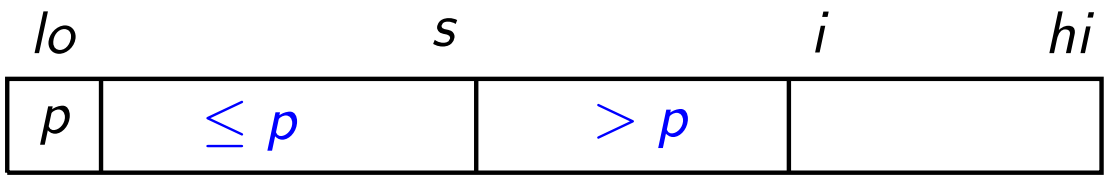
Lomuto Partitioning

s: the index of the *last seen* element $\leq p$ so far; initially $s \leftarrow lo$

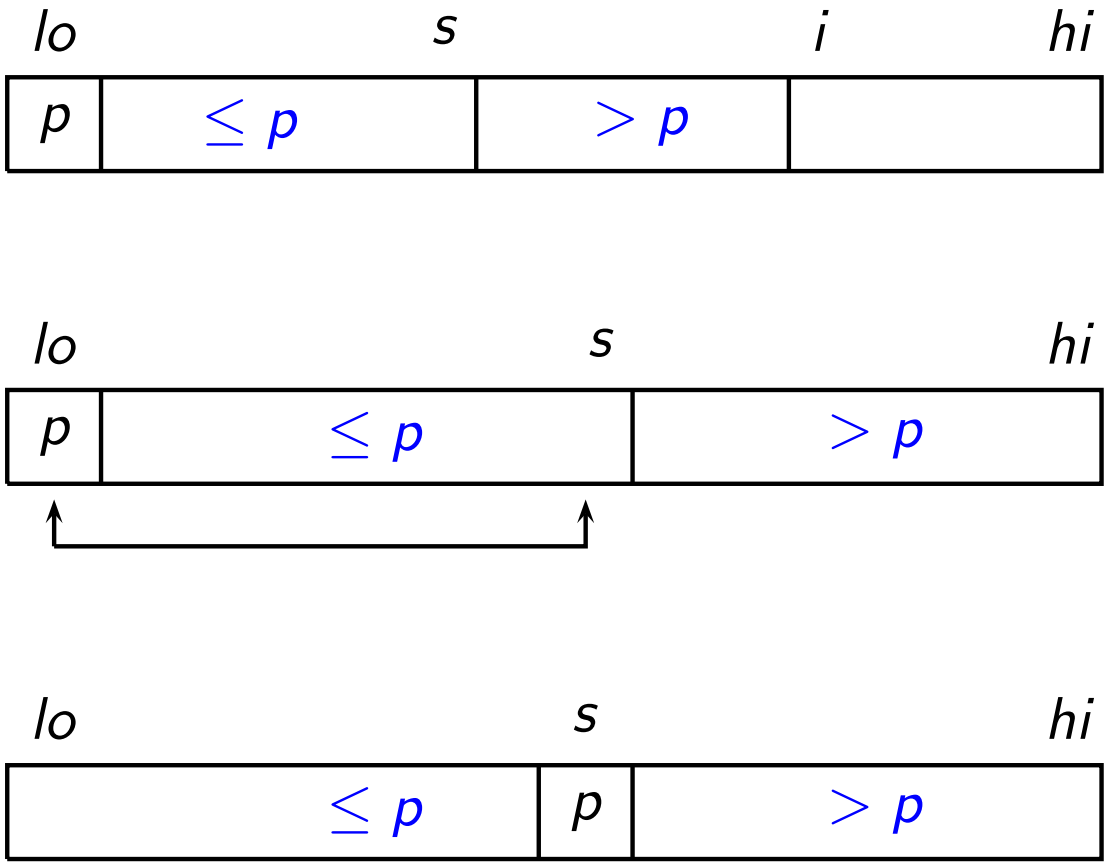
i: the index of the *next unseen* element; initially, $i \leftarrow lo + 1$

The **Lomuto Partitioning** algorithm works as follows:

- $p \leftarrow A[lo]$; $s \leftarrow lo$;
- for $i \leftarrow lo + 1$ to hi, do the following:
  - if $A[i] \leq p$, then $s \leftarrow s + 1$, and $swap(A[s], A[i])$;
- $swap(A[lo], A[s])$;
- return s as the index of the pivot p;



Lomuto Partitioning



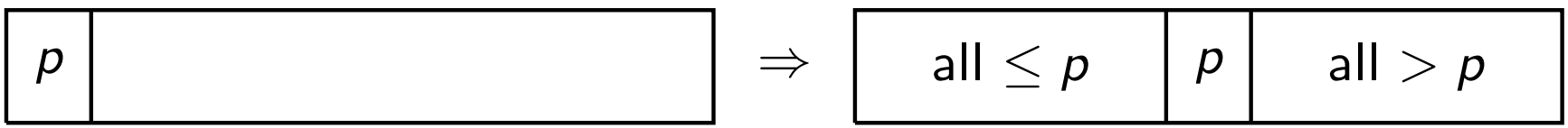
Lomuto Partitioning: Example

s	*i*								
72	29	64	86	33	89	38	32	94	42
	s	*i*							
72	29	64	86	33	89	38	32	94	42
		s		*i*					
72	29	64	86	33	89	38	32	94	42
			s			*i*			
72	29	64	33	86	89	38	32	94	42
				s			*i*		
72	29	64	33	38	89	86	32	94	42
					s				*i*
72	29	64	33	38	32	86	89	94	42
						s			*i*
72	29	64	33	38	32	42	89	94	86
						s			*i*
42	29	64	33	38	32	72	89	94	86

Finding the kth Smallest Element

Here is how we can use partitioning to find the kth smallest element.

function QuickSelect($A[\cdot], lo, hi, k$)
    $s \leftarrow$ LomutoPartition(A, lo, hi)
    if $s - lo + 1 = k$ **then**
        return $A[s]$
    else
        if $s - lo + 1 > k$ **then**
            QuickSelect($A, lo, s - 1, k$)
        else
            QuickSelect($A, s + 1, hi, k - (s - lo + 1)$)



Example: Finding the 5th Smallest Element

s	i								
72	29	64	86	33	89	38	32	94	42
						s			i
42	29	64	33	38	32	72	89	94	86

As $s = 7$ and $k = 5$, assuming $lo = 1$, so $s - lo + 1 = 7 > k$, repeat the process on the left part $A[1, s-1]$, that is, $lo = 1, hi = s - 1 = 6, k = 5$:

42	29	64	33	38	32

42 being the new pivot.

Example: Finding the 5th Smallest Element

s	*i*				
42	29	64	33	38	32
	s		*i*		
42	29	64	33	38	32
		s		*i*	
42	29	33	64	38	32
			s		*i*
42	29	33	38	64	32
				s	*i*
42	29	33	38	32	64

Finally the pivot is swapped into its correct position, which happens to be the fifth (index 5):

				s	*i*
32	29	33	38	42	64

Hence, QuickSelect returns 42.

Question: What is the worst-case time complexity of the above QuickSelect algorithm?

QuickSelect Algorithm: Analysis

In the QuickSelect algorithm, in each iteration, we first partition the input array into three parts: $A[lo, s-1]$, $A[s]$ and $A[s+1, hi]$, and recur into either the left or right part if necessary.

Clearly, in each iteration, the cost of partitioning is *linear* to the length of the current array $A[lo, hi]$, i.e., $O(|A[lo, hi]|)$.

Furthermore, in the worst case with a **very bad pivot** p, the array length can only be **decreased by** 1.

Therefore, the overall worst-case running time cost is bounded by:

$$C(n) = n + (n-1) + (n-2) + \ldots + 1 = \frac{n(n-1)}{2} \in O(n^2)\,.$$

QuickSelect Algorithm: Analysis (II)

Interestingly, if we can find a *"good"* pivot p in each iteration such that:

- both the lengths of the left and right parts with respect to p are $\leq \frac{2}{3} \cdot |A[lo, hi]|$, namely, 2/3 of the length of the current array $A[lo, hi]$,

then, in each iteration, the problem size can be reduced by a constant factor of 2/3.

If this is the case, the overall running time cost becomes:

$$C(1) = 1$$

$$C(n) \leq C(\frac{2}{3} \cdot n) + n\,.$$

Expanding the recursion, we have:

$$C(n) \leq n + \frac{2}{3} \cdot n + (\frac{2}{3})^2 \cdot n + \ldots + (\frac{2}{3})^i \cdot n + \ldots \leq \frac{n}{1 - \frac{2}{3}} = 3 \cdot n \in O(n)\,.$$

Randomized QuickSelect Algorithm

By adopting randomization, we can find an aforementioned good pivot p in $O(|A[lo, hi]|)$ **time in expectation**, i.e., linear expected time to the length of the current array $A[lo, hi]$.

As a result, the **expected** running time cost of each iteration is bounded by $O(|A[lo, hi]|)$.

Hence, the overall running time cost is bounded by $O(n)$ in expectation.

Randomized QuickSelect Algorithm: Analysis

A good pivot will be an element in the middle one-third range (sorted) of $A[lo, hi]$.

The probability of selecting a good pivot is at least $1/3$.

In expectation, in at most 3 trials, we can find a good pivot.

As each trial takes $O(|A[lo, hi]|)$ time for partitioning, in expectation, each iteration takes $O(|A[lo, hi]|)$ time.

Therefore, the overall running time of **Randomized QuickSelect** is bounded by $O(n)$ in expectation.

Randomized QuickSelect Algorithm

The implementation of this randomization trick is as follows:

In each iteration of QuickSelect,

- select a pivot p uniformly at random from the current array $A[lo, hi]$;
- partition $A[lo, hi]$ with p;
- if both the left part $A[lo, s-1]$ and the right part $A[s+1, hi]$ have lengths at most $\frac{2}{3} \cdot |A[lo, hi]|$, then accept pivot p and enter to the next iteration;
- otherwise, **repeat** from the first step by selecting another pivot uniformly at random from $A[lo, hi]$.